

Academic Project Report: Interactive Python Quiz Portal

Welcome to the project report for the Interactive Python Quiz Portal. Built entirely using Python and the json file which saves the codes and the database via which the output is coming

1. Project Overview & Features

This project uses a lot of interfaces along with a large [no](#). Of modules which makes the codes to work smoothly

- **Role-Based Access:** A primary entry menu safely routes users into either the Teacher portal or the students portal which makes it user friendly
- **Teacher Capabilities:** Teachers can add new questions to the database by entering question strings, specifying up to 4 multiple-choice options, and flagging the correct answer index.
- **Student Experience:** Students initiate their session by registering their name. They navigate through multiple-choice questions by choosing numbers 1–4, receiving immediate feedback ("Correct ans" or "Wrong ans") after each submission.
- **Performance Tracking:** Upon quiz completion, the application calculates total scores and persists them with a timestamp. Students can inspect past performance through the "My Scores" lookup feature.
- **Persistent Data:** All the data is saved automatically

2. System Architecture & Flowcharts

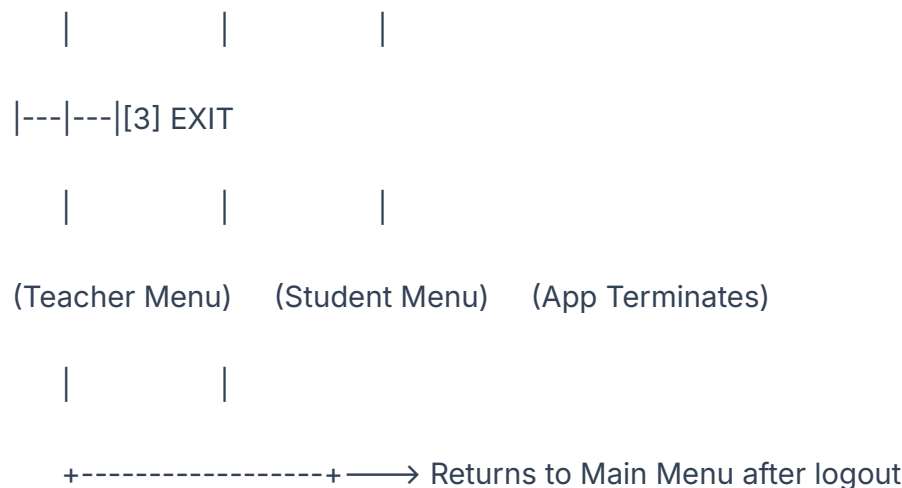
To ensure a structured user experience, the system utilizes nested control loops. The workflows below illustrate navigation through the primary application modules.

Core Application Flow

[RUN main.py] —→ [MAIN MENU]

|

+-----+-----+



Teacher Workflow (**teacher.py**)

[Teacher Menu]

|-- [1] Add Question → Enter Text → Enter Options (1-4) → Select Correct Answer → Save to JSON DB

|-- [2] Remove Question → Enter Question ID

`-- [3] Logout -----→ Returns to Main Menu

Student Workflow (**student.py**)

[Student Menu]

|-- [1] Start Quiz --→ Enter Name → Answer Loop → Score Calculation
→ Save Score & Timestamp

|-- [2] My Scores ----→ Input Name → Query History & Display Timestamps

`-- [3] Logout -----→ Returns to Main Menu

3. Module Breakdown & Code Structure

The code base is partitioned into standalone Python modules to ensure maintainability and separation of concerns.

Module	Primary Responsibility	Key Functions & Logic
<code>main.py</code>	Application Entry Point	Initializes database via <code>db.init_db()</code> and executes primary control loop for routing user roles.
<code>db.py</code>	Storage Interface	Interfaces with <code>json</code> and <code>os</code> libraries. Manages reading and writing to <code>quiz_data.json</code> .
<code>teacher.py</code>	Administration Panel	Contains administrative routines for managing test content, including question creation loops.
<code>student.py</code>	Examination Engine	Controls test execution, handles answer validation via <code>.isdigit()</code> , and computes final metrics.
<code>quiz_data.json</code>	Persistence Storage	Structured data file storing question schema and player performance records.

Module Details

A. Entry Point: `main.py`

This module startups the initial application and triggering database setup via `db.init_db()`. An infinite execution loop (`while True:`) presents the primary menu choices (1. `Teacher`, 2. `Student`, 3. `EXIT`) until explicitly terminated by the user.

B. Database Manager: `db.py`

Built using `json` and `os` modules to maintain zero external dependencies. The `init_db()` routine inspects local storage for `quiz_data.json`, creating an empty structural template with `questions` and `scores` arrays if missing. Helper routines

including `get_questions()`, `add_question()`, `save_score()`, and `get_scores()` wrap input/output routines to ensure consistent schema handling.

C. The Admin Panel: `teacher.py`

Executes the teacher control loop (1. Add Question, 2. Remove Question, 3. Logout). The `add ques()` logic reads question inputs, steps through option collection, and captures the correct index (adjusting by -1 to normalize against 0-indexed Python arrays) before dispatching updates to `db.add_question`.

D. The Gameplay Engine: `student.py`

Retrieves test sets using `db.get_questions()`. Upon validating question existence, it iterates using `enumerate()`, presenting options structured 1 to 4. Numerical input checks using `answer.isdigit()` against non-numeric. Final performance are calculated against total questions (`len(ques)`) and recorded to history.

E. Storage Medium: `quiz_data.json`

Human-readable JSON document storing array structures for content and metrics:

- **Questions:** Objects containing `question` text, an `options` list, and an `answer` zero-based index integer.
- **Scores:** Objects tracking `name`, calculated `score`, total `questions`, and an automated `played_at` timestamp string.

4. Future Enhancements

Future revisions of the Interactive Python Quiz Portal will focus on the following features:

1. **Complete Question Deletion:** Fully implement the `remove_que()` handler within `teacher.py` to allow instructors to prune out-of-date questions from the active database.
2. **Refine Option Input Flow:** Refactor prompt sequencing in `add_ques()` so that answer key selection takes place cleanly after all four option strings are collected.
3. **Authentication Layer:** Implement secure access credentials for the Teacher panel to restrict unauthorized modifications to quiz content.